UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA.

VICERRECTORADO ACADÉMICO.

COORDINACIÓN GENERAL DE PREGRADO.

PROYECTO DE CARRERA: INGENIERÍA EN INFORMÁTICA

BASE DE DATOS II.

Proyecto de Optimización de Rutas Logísticas con Grafos

PROFESOR:

Cordero Clinia

ALUMNOS:

Rodriguez Edfrank CI 30.857.264
Palma Franyibeth CI 31.564.549
Muñoz Elishama CI 31.461.662

CIUDAD GUAYANA; MARZO DE 2026.

Optimización de Rutas Logísticas con Grafos (Neo4j + Graph Data Science)

El presente proyecto tiene como objetivo desarrollar un sistema de optimización de rutas logísticas utilizando una base de datos orientada a grafos. Para ello se implementa Neo4j, permitiendo representar nodos como almacenes, intersecciones y puntos de entrega, así como las rutas que los conectan.

Mediante el uso de la librería Graph Data Science (GDS) se aplican algoritmos de grafos como Dijkstra y A*, los cuales permiten calcular rutas óptimas considerando factores como distancia, tiempo estimado y condiciones de tráfico. De esta manera, el sistema permite analizar y seleccionar las rutas más eficientes dentro de una red de transporte.

1. Creación de Índices para Optimización de Consultas

```
// 1. Crear Índices para optimizar el rendimiento
CREATE INDEX IF NOT EXISTS FOR (n:Almacen) ON (n.id);
CREATE INDEX IF NOT EXISTS FOR (n:PuntoEntrega) ON (n.id);
CREATE INDEX IF NOT EXISTS FOR (n:Interseccion) ON (n.id);
```

Este código en Cypher crea índices en una base de datos Neo4j para optimizar el rendimiento de las consultas. Específicamente, genera índices sobre la propiedad "id" para tres tipos de nodos: Almacen, PuntoEntrega e Interseccion, utilizando la cláusula "IF NOT EXISTS" para evitar errores si los índices ya existen previamente, lo que acelera significativamente las búsquedas y filtrados por este identificador en consultas futuras.

2. Carga y Creación de Nodos del Sistema Logístico

```
// 2. Cargar Nodos (Almacenes, Intersecciones, Puntos de Entrega)
UNWIND [
  {id: 'A1', nombre: 'Almacén Central', tipo: 'Almacen'},
  {id: 'I1', nombre: 'Intersección Norte', tipo: 'Interseccion'},
  {id: 'I2', nombre: 'Intersección Sur', tipo: 'Interseccion'},
  {id: 'I3', nombre: 'Intersección Este', tipo: 'Interseccion'},
  {id: 'PE1', nombre: 'Cliente A', tipo: 'PuntoEntrega'},
  {id: 'PE2', nombre: 'Cliente B', tipo: 'PuntoEntrega'},
  {id: 'PE3', nombre: 'Cliente C', tipo: 'PuntoEntrega'}
] AS nodo
CALL {
  WITH nodo
  MERGE (n:Almacen {id: nodo.id}) WHERE nodo.tipo = 'Almacen'
  MERGE (n:Interseccion {id: nodo.id}) WHERE nodo.tipo = 'Interseccion'
  MERGE (n:PuntoEntrega {id: nodo.id}) WHERE nodo.tipo = 'PuntoEntrega'
  SET n.nombre = nodo.nombre
} IN TRANSACTIONS OF 100 ROWS;
```

Este código en Cypher carga nodos en Neo4j a partir de una lista de datos. Primero, utiliza UNWIND para descomponer un arreglo que contiene siete elementos con propiedades como id, nombre y tipo, representando almacenes, intersecciones y puntos de entrega. Luego, mediante una subconsulta con CALL, evalúa el tipo de cada nodo y utiliza MERGE para crearlo con la etiqueta correspondiente (Almacen, IntersecciOn o PuntoEntrega) basándose en su id, evitando duplicados si ya existe. Finalmente, asigna la propiedad nombre a cada nodo y procesa la operación en transacciones de 100 filas para optimizar el rendimiento.

3. Creación de Relaciones y Rutas entre Nodos

```
// 3. Cargar Relaciones (Rutas entre nodos)
UNWIND [
  {origen: 'A1', destino: 'I1', distancia: 10, tiempo_estimado: 15, nivel_trafico: 0.2, peso_maximo: 50},
  {origen: 'A1', destino: 'I2', distancia: 15, tiempo_estimado: 20, nivel_trafico: 0.5, peso_maximo: 30},
  {origen: 'I1', destino: 'I3', distancia: 5, tiempo_estimado: 8, nivel_trafico: 0.1, peso_maximo: 50},
  {origen: 'I1', destino: 'PE1', distancia: 8, tiempo_estimado: 12, nivel_trafico: 0.8, peso_maximo: 20},
  {origen: 'I2', destino: 'I3', distancia: 12, tiempo_estimado: 18, nivel_trafico: 0.6, peso_maximo: 40},
  {origen: 'I3', destino: 'PE2', distancia: 6, tiempo_estimado: 10, nivel_trafico: 0.3, peso_maximo: 50},
  {origen: 'I3', destino: 'PE3', distancia: 9, tiempo_estimado: 14, nivel_trafico: 0.4, peso_maximo: 45}
] AS ruta
MATCH (o:Almacen {id: ruta.origen}), (d:Interseccion {id: ruta.destino})
MERGE (o)-[r:CONECTA_A]->(d)
SET r.distancia = ruta.distancia,
    r.tiempo_estimado = ruta.tiempo_estimado,
    r.nivel_trafico = ruta.nivel_trafico,
    r.peso_maximo = ruta.peso_maximo;
```

En este código carga relaciones entre nodos en Neo4j a partir de una lista de rutas. Utiliza UNWIND para procesar un arreglo que contiene siete relaciones con propiedades como origen, destino, distancia, tiempo estimado, nivel de tráfico y peso máximo. Para cada

elemento, busca el nodo de origen con la etiqueta Almacen y el nodo de destino con la etiqueta Interseccion utilizando sus respectivos identificadores, y crea o actualiza mediante MERGE una relación dirigida llamada CONECTA_A entre ellos. Finalmente, asigna todas las propiedades de la ruta a la relación creada, estableciendo así la conectividad y características del camino entre los nodos.

Consultas de Optimización y Análisis de Rutas

Consulta 1. Búsqueda de la Ruta Más Corta por Distancia

```
// Consulta 1: Ruta más corta por Distancia
MATCH (start:Almacen {id: 'A1'}), (end:PuntoEntrega {id: 'PE1'})
MATCH path = shortestPath((start)-[:CONECTA_A*]->(end))
WITH path, reduce(total = 0, r IN relationships(path) | total + r.distancia) AS distancia_total
RETURN [n IN nodes(path) | n.id] AS ruta, distancia_total;
```

Esta consulta encuentra la ruta más corta por distancia entre el Almacén Central y el Cliente
A, considerando el número de conexiones. Primero localiza el nodo de origen con id 'A1' y el nodo destino con id 'PE1', luego busca el camino con menos relaciones CONECTA_A entre ellos usando la función shortestPath. Después calcula la distancia total sumando los valores de distancia de cada relación en ese camino mediante reduce, y finalmente muestra los identificadores de los nodos que forman la ruta junto con la distancia total acumulada**.**

Consulta 2. Cálculo de la Ruta Más Rápida Basada en Tiempo Estimado

```
// Consulta 2: Ruta más rápida por Tiempo
MATCH (start:Almacen {id: 'A1'}), (end:PuntoEntrega {id: 'PE1'})
MATCH path = shortestPath((start)-[:CONECTA_A*]->(end))
WITH path, reduce(total = 0, r IN relationships(path) | total + r.tiempo_estimado) AS tiempo_total
RETURN [n IN nodes(path) | n.id] AS ruta_rapida, tiempo_total;
```

Esta consulta encuentra la ruta más rápida basada en el tiempo estimado entre el Almacén Central y el Cliente A. Primero localiza el nodo de origen con id 'A1' y el nodo destino con id 'PE1', luego busca el camino con menos relaciones CONECTA_A entre ellos mediante la función shortestPath. Después calcula el tiempo total acumulado sumando los valores de tiempo_estimado de cada relación en ese recorrido utilizando reduce, y finalmente devuelve los identificadores de los nodos que componen la ruta junto con el tiempo total estimado del viaje.

Consulta 3. Cálculo de Costo Dinámico Considerando Tráfico

```
// Consulta 3: Costo Dinámico (Distancia × Tráfico)
MATCH (start:Almacen {id: 'A1'}), (end:PuntoEntrega {id: 'PE1'})
MATCH path = shortestPath((start)-[:CONECTA_A*]->(end))
WITH path, reduce(totalCosto = 0, r IN relationships(path) | totalCosto + (r.distancia * (1 + r.nivel_trafico))) AS costo_total
RETURN costo_total;
```

Esta consulta calcula un costo dinámico basado en la distancia afectada por el nivel de tráfico entre el Almacén Central y el Cliente A. Primero localiza el nodo de origen con id 'A1' y el nodo destino con id 'PE1', luego busca el camino con menos relaciones CONECTA_A entre ellos usando la función shortestPath. Después calcula el costo total acumulado sumando para cada relación el producto de su distancia multiplicada por uno más su nivel de tráfico, utilizando la función reduce, y finalmente devuelve únicamente el valor del costo total resultante.

Consulta 4. Búsqueda de Ruta con Restricción de Capacidad de Carga

```
// Consulta 4: Restricción de Carga (Filtrado)
MATCH (start:Almacen {id: 'A1'}), (end:PuntoEntrega {id: 'PE2'})
MATCH path = shortestPath((start)-[:CONECTA_A*]->(end))
WHERE ALL(r IN relationships(path) WHERE r.peso_maximo >= 40)
RETURN [n IN nodes(path) | n.id] AS ruta_valida;
```

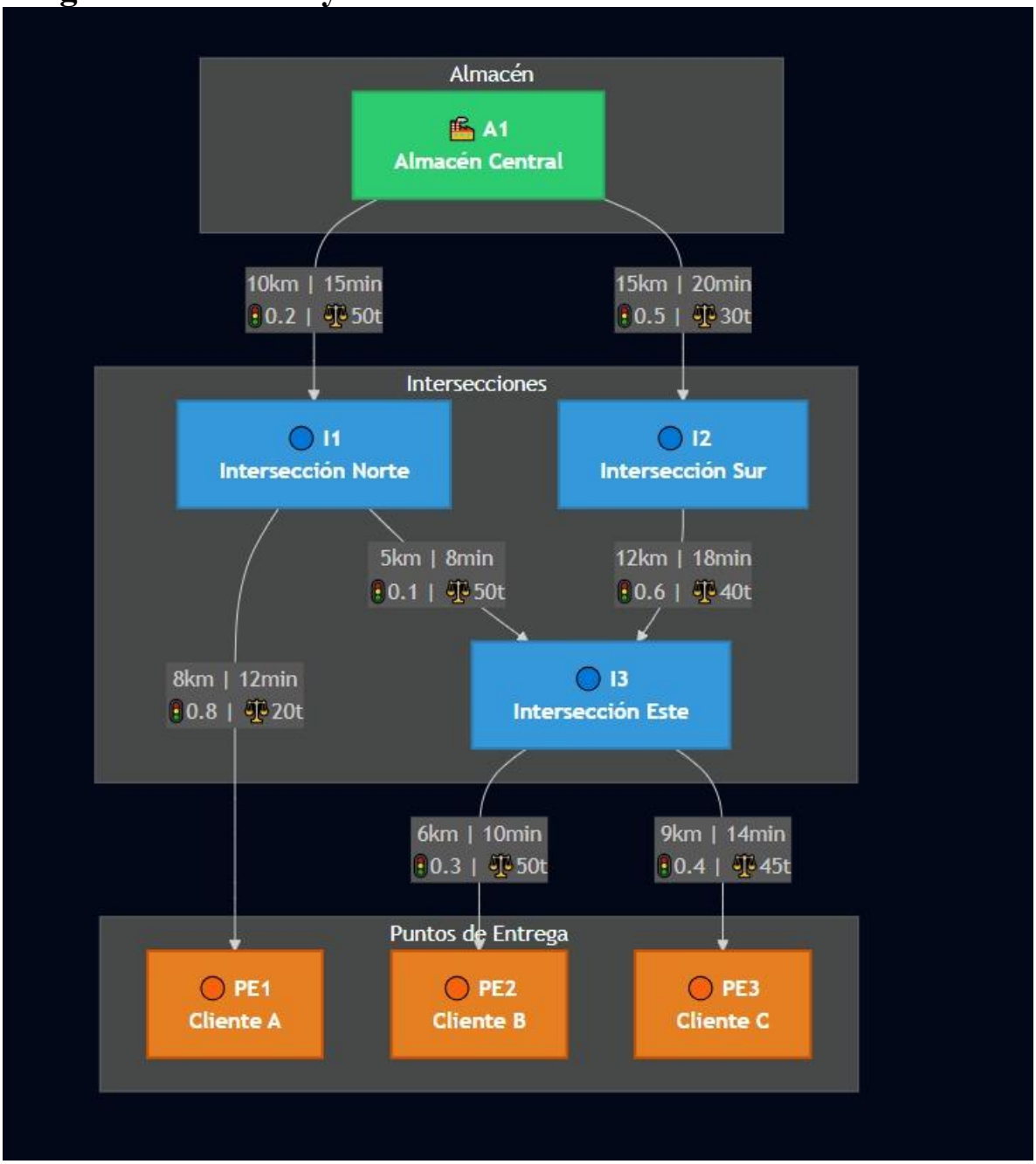
Esta consulta encuentra una ruta válida entre el Almacén Central y el Cliente B que cumple con una restricción de carga mínima. Primero localiza el nodo de origen con id 'A1' y el nodo destino con id 'PE2', luego busca el camino con menos relaciones CONECTA_A entre ellos mediante la función shortestPath. Después aplica un filtro con WHERE que verifica que todas las relaciones en el camino tengan un peso_maximo mayor o igual a 40, utilizando la condición ALL, y finalmente devuelve únicamente los identificadores de los nodos que conforman la ruta que cumple con este requisito.

Consulta 5. Exploración de Rutas con Profundidad Variable

```
// Consulta 5: Multi-Parada (Expandir camino)
MATCH (n:Almacen {id: 'A1'})
MATCH path = (n)-[:CONECTA_A*1..10]->()
RETURN path;
```

Esta consulta explora rutas de múltiples paradas desde el Almacén Central con una profundidad variable. Primero localiza el nodo con etiqueta Almacen y id 'A1', luego busca todos los caminos que parten desde este nodo siguiendo relaciones CONECTA_A entre 1 y 10 niveles de profundidad, sin importar el tipo de nodo de destino. Finalmente devuelve todos los caminos encontrados dentro de ese rango de expansión, permitiendo visualizar las posibles rutas que se desprenden desde el almacén.

Diagrama de nodos y relaciones



Aquí se poder ver cómo se organiza visualmente los elementos del sistema Logistics Optimizer, mostrando los tipos de nodos existentes, los nodos específicos con sus identificadores y nombres, y las conexiones que parten desde cada uno de ellos con sus respectivas propiedades.

En la parte superior se encuentra el Almacén Central identificado como A1, del cual salen dos conexiones: una hacia la Intersección Norte I1 con 10 kilómetros de distancia, 15

minutos de tiempo estimado, nivel de tráfico de 0.2 y peso máximo de 50 toneladas, y otra hacia la Intersección Sur I2 con 15 kilómetros, 20 minutos, tráfico de 0.5 y peso máximo de 30 toneladas. En el siguiente nivel se presentan las intersecciones, destacando la Intersección Norte I1, desde donde parten cuatro conexiones: hacia el Cliente A PE1 con 8 kilómetros, 12 minutos, tráfico de 0.8 y peso máximo de 20 toneladas; hacia la Intersección Este I3 con 5 kilómetros, 8 minutos, tráfico de 0.1 y peso máximo de 50 toneladas; hacia el Cliente B PE2 con 6 kilómetros, 10 minutos, tráfico de 0.3 y peso máximo de 50 toneladas; y hacia el Cliente C PE3 con 9 kilómetros, 14 minutos, tráfico de 0.4 y peso máximo de 45 toneladas.

Finalmente, en la parte inferior se listan los puntos de entrega que corresponden a los clientes PE1, PE2 y PE3, completando así la representación estructurada de todos los nodos y relaciones del sistema, donde cada elemento tiene su lugar en la jerarquía y se especifican las características de las rutas disponibles desde cada punto de origen.

Dashboard

En el panel Mapa de Red se observa la representación del grafo utilizado en el proyecto. Allí se muestran los diferentes nodos que representan el almacén (A1), las intersecciones (I1, I2, I3) y los puntos de entrega (PE1, PE2, PE3), junto con las conexiones que representan las rutas entre ellos.

El apartado Análisis de Tráfico Crítico presenta un gráfico que muestra el nivel de tráfico asociado a distintos clientes o rutas, lo que ayuda a identificar posibles zonas donde el tráfico puede afectar la eficiencia de las entregas.

También se muestra el Costo Total Estimado, el cual representa el valor calculado de la ruta seleccionada considerando factores como la distancia y las condiciones de tráfico.

Finalmente, el dashboard incluye controles para seleccionar el origen y destino de la ruta, además de tablas que muestran los resultados de las consultas, como la ruta encontrada y su distancia total, así como información general sobre el estado de la red.

Justificación: por qué neo4j es mejor que sql para logistics optimizer

Para el desarrollo del proyecto Logistics Optimizer, se seleccionó Neo4j como sistema de base de datos en lugar de una base de datos relacional tradicional. Esto se debe a que el problema que se busca resolver consiste en encontrar rutas óptimas entre almacenes y puntos de entrega, considerando factores como la distancia, el tráfico y la capacidad de carga de las rutas. Este tipo de problema se adapta de forma natural a un modelo de grafos, donde las intersecciones y ubicaciones se representan como nodos y las calles como conexiones entre ellos.

Si se hubiera utilizado una base de datos SQL, habría sido necesario crear múltiples tablas separadas para almacenar almacenes, intersecciones, puntos de entrega y calles, relacionándolas mediante claves foráneas. Para calcular rutas que atraviesen varios puntos, las consultas tendrían que incluir numerosos JOINs entre tablas, lo que aumentaría considerablemente la complejidad y el tiempo de ejecución de las consultas, especialmente a medida que la red crece.

En cambio, con Neo4j la red vial puede modelarse de forma directa e intuitiva: cada ubicación se representa como un nodo y cada calle como una relación entre nodos. Esto permite realizar consultas de rutas de manera mucho más sencilla, ya que solo es necesario recorrer las conexiones entre nodos para encontrar caminos posibles entre dos puntos. Como resultado, las consultas son más claras, más cortas y más fáciles de mantener.

Otro aspecto importante es que el proyecto requiere utilizar algoritmos de optimización de rutas como Dijkstra y A*. Implementar estos algoritmos en un sistema SQL implicaría desarrollarlos manualmente dentro de la aplicación, lo cual aumentaría la complejidad del sistema. Neo4j, por su parte, ofrece la librería Graph Data Science, que incluye estos algoritmos ya optimizados, permitiendo ejecutarlos de forma directa sobre el grafo.

Además, el sistema necesita calcular costos dinámicos de las rutas, tomando en cuenta variables como la distancia y el nivel de tráfico, así como aplicar restricciones relacionadas con la capacidad de carga de los vehículos. En un modelo de grafos estas operaciones se realizan de manera natural al recorrer las relaciones entre nodos y evaluar sus propiedades, mientras que en SQL este tipo de cálculos puede complicar significativamente las consultas.

Finalmente, el uso de Neo4j también ofrece ventajas en términos de escalabilidad y flexibilidad. Si en el futuro se desea ampliar el sistema incorporando nuevos tipos de ubicaciones o rutas, simplemente se pueden agregar nuevos nodos y relaciones al grafo sin

necesidad de rediseñar la estructura existente. Esto permite que el sistema evolucione de forma más sencilla a medida que crece la red logística.

Repositorio: https://github.com/edfranklra-art/PROYECTODB2.git